Workflow and Project Management

USP 410/510 | Spring 2026

Dr. Liming Wang

Portland State University

Part 1: Why Workflow Matters

Based on Yu and Barter, Chapter 3, and Turrell workflow chapters

Today

- How workflow choices affect quality and speed
- A practical project structure for this class
- Reproducibility essentials (data, code, environment)
- Git and collaboration basics for course projects
- In-class demo: from empty folder to reproducible project

Core Claim

Good analysis with weak workflow is fragile

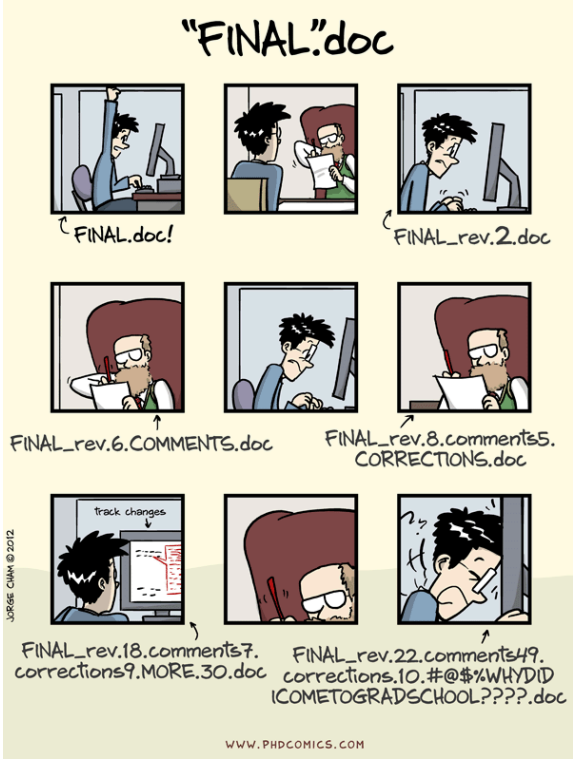
- Hard to reproduce
- Hard to debug
- Hard to extend
- Hard to collaborate on

Workflow is not overhead. It is risk management.

Symptoms of Workflow Debt

If you have seen these, your workflow needs improvement:

- `final_analysis.ipynb`, `final_analysis_v2.ipynb`, `final_final.ipynb`
- Unsure which file generated the figure in your report
- Raw data files edited manually and silently
- "Works on my computer" but fails for teammates
- No clear record of what changed and why



Source: PHD Comics

A Practical Workflow Loop

1. **Define** question and scope
2. **Structure** project files and environment
3. **Implement** in small, testable steps
4. **Document** decisions and assumptions
5. **Review** outputs and code
6. **Share** and iterate

This loop repeats every week of your project.

Part 2: Project Setup (VDS Chapter 3)

Consistent Project Structure

Yu and Barter emphasize team-agreed structure plus clear documentation.

```
my_project/
 README.md
 data/
   raw/
   processed/
 code/
   01_cleaning.ipynb
   02_eda.ipynb
   03_modeling.ipynb
   functions/
     cleaning.py
 outputs/
   figures/
   tables/
```

Why Structure Helps

- Faster onboarding for teammates
- Less time hunting for files
- Easier handoff between analysis stages
- Easier reproduction when grading or presenting
- Better long-term maintainability

README as Project Contract

Your `README.md` should answer:

- What question is this project answering?
- Where is raw data from?
- How do I run the analysis?
- What dependencies are required?
- Where are final outputs located?

If a classmate cannot run your project from README, it is incomplete.

Raw Data Rule

Do not manually edit original raw files.

- Keep `data/raw/` unchanged (treat the folder as `read-only`)
- Put all cleaning logic in code
- Re-run cleaning when source data or logic changes
- If you must cache cleaned data, document exactly how it was created

Functions for Cleaning Decisions

VDS recommends reusable cleaning functions with explicit arguments.

```python
# code/functions/cleaning.py
import pandas as pd


def clean_crashes(df: pd.DataFrame, drop_missing_loc: bool = True) -> pd.DataFrame:
    out = df.copy()
    out.columns = out.columns.str.strip().str.upper()
    if drop_missing_loc:
        out = out.dropna(subset=["LATITUDE", "LONGITUDE"])
    out["CRASH_DT"] = pd.to_datetime(out["CRASH_DT"], errors="coerce")
    return out
```

Reproducibility: Weak vs Strong

- **Weak**: rerunning same code on same data reproduces same result
- **Stronger**: others can run it on another machine and get same result
- **Strongest in practice**: findings are stable to reasonable alternative choices

Reproducible does not automatically mean correct.

Habits That Improve Trustworthiness

- Small functions with clear inputs and outputs
- Frequent reruns from top to bottom
- Code review (peer or self-review)
- Explicit assumptions in markdown cells/comments
- Save outputs with deterministic names

Part 3: Workflow Chapters (Turrell)

Scripts vs Notebooks vs Quarto

FORMAT	BEST FOR	MAIN TRADEOFF
`.py` scripts	Reusable analysis pipelines	Less narrative context
`.ipynb` notebooks	Exploration + teaching + narrative	Messier version control
`.qmd` Quarto	Publication-ready reproducible docs	Slightly more setup

Turrell's typical recommendation: scripts + interactive window.

Typical Coding Workflow

1. Open project folder in Antigravity
2. Write code in script(s)
3. Run selected lines interactively (`Shift+Enter`)
4. Install packages as needed
5. Promote stable code into functions/modules
6. Export results to report/notebook/dashboard

Style Rules That Prevent Bugs

From workflow style chapters:

- Use descriptive snake_case names
- Keep comments focused on **why**, not line-by-line **what**
- Use formatter/linter tools for consistency
- Keep functions focused (one job)
- Apply DRY, KISS, and Separation of Concerns

Packages and Environments

Use isolated environments per project.

```
uv init
uv add pandas jupyter matplotlib seaborn
uv run python --version
uv pip list
```

Track dependencies in `pyproject.toml` so others can reproduce your setup.

Workflow Management Tools

Potential packages to explore for workflow management:

- `perfect` (Python)
- `snakemake` (Python)
- `targets` ®

These tools help formalize multi-step pipelines, dependencies, and reruns.

Environment Checklist for This Class

- One environment per course project
- Dependencies declared in `pyproject.toml`
- Notebook kernel points to project `.venv`
- No hidden dependency installed globally only on your laptop

Part 4: Collaboration and Project Management

Minimal Git Workflow

```
git init
git add .
git commit -m "Initial project scaffold"
# edit files
git add .
git commit -m "Add cleaning function and EDA notebook"
```

For shared projects: pull before pushing, commit often, write meaningful messages.

Collaboration Norms for Team Projects

- Agree on folder structure early
- Define file ownership to reduce merge conflicts
- Use issues or a task list to track work
- Review each other’s code before major merges
- Record key decisions in README or changelog

AI-Assisted Workflow (Course Policy Aligned)

AI can accelerate coding, but accountability stays with you.

- Ask AI for drafts, refactors, debugging support
- Verify with tests, row counts, and sanity checks
- Keep prompts and assumptions documented for important steps
- Never submit code you cannot explain

A1 Reality Check (Due Today)

Assignment 1 due date: **Thursday, April 23, 2026**

Before submitting, confirm:

- Notebook or Quarto document runs top-to-bottom
- Visualizations have labeled axes/units
- Narrative answers your research question clearly
- AI use statement is included (if used)

Part 5: In-Class Demo

Demo Goal

Build a reproducible crash-analysis project in about 15 minutes:

1. Scaffold project structure
2. Initialize `uv` environment
3. Add minimal cleaning function
4. Create starter notebook/script
5. Capture first Git commit

Demo Files

Use the companion runbook:

week4_demo.md

It includes command-by-command instructions and a complete mini workflow.

In-Class Exercise (10-15 min)

Individually or in pairs:

1. Create your own project scaffold
2. Write one cleaning function with at least one argument
3. Create a README with run instructions
4. Make one commit with a clear message

Optional stretch: add a second commit that refactors your function.

Common Failure Modes to Avoid

- Mixing raw and processed data in one folder
- Copy/paste code blocks instead of making functions
- Keeping all analysis in one huge notebook
- Committing only once at the end
- Missing README instructions

Key Takeaways

- Project management is part of data quality
- Structure + documentation reduce errors and rework
- Reproducibility needs data, code, and environment discipline
- Simple workflows beat clever chaotic workflows
- Start clean now to save time in Weeks 7-11

For Next Week

Week 5 topic (Thursday, April 30, 2026): Exploring and visualizing data

Read:

- Yu and Barter, Chapter 5: Exploratory Data Analysis
- Turrell, Visualize chapter

Due by Thursday, April 30, 2026:

- DataCamp DC2: Data Manipulation with pandas

Week 4 sources:

- Yu and Barter, Chapter 3: Setting Up Your Data Science Project
- Turrell, Workflow chapters